

Spike: 7**Title:** Agent Marksmanship**Author:** Pattarapol Tantechasa, 103883220**Goals / deliverables:****Expected Outcomes (Baseline Task)**

1. A fully functional code simulation demonstrating the four specified weapon profiles (Rifle, Rocket, Hand Gun, Hand Grenade).
2. Visual proof that the attacking agent successfully calculates interception points to hit a moving target.
3. A comprehensive Spike Report detailing your design decisions, the mathematics used for target prediction, and any challenges faced.

Expected Outcomes (from extensions)

1. Code demonstrating an agent dynamically closing the distance to a target before firing a programmed shotgun weapon.
2. Implementation of radius-based collision/damage application upon projectile detonation, with the AI intentionally targeting the proximity rather than a direct body hit.
3. Enhanced steering behaviours on the target agent, showcasing evasion logic (fleeing incoming slow projectiles) and obstacle avoidance/cover-seeking logic.

Technologies, Tools, and Resources used:

- Claude: brainstorm ideas, explain code and debugs.
- Visual Studio Code IDE
- Conda: Python environment manager

Tasks undertaken:

1. Download and Install **Visual Studio Code**
2. Install **Python** and **Pyglet** on your Machine
3. Open the main project downloaded
4. Open terminal and run cd command : **cd Portfolio-Tasks/Portfolio-Task-15-Spike-Agent-Marksmanship**
5. For baseline version : **cd marksmanship-baseline**
6. For extended version : **cd marksmanship -extensions**
7. Run : **python main.py**

Design Phase: Agent Marksmanship

Overview

The goal of this spike was to explore how an attacking agent can intelligently aim at a moving target by predicting where the target will be when a projectile arrives, rather than simply shooting at its current position. I built a system where a static attacker fires projectiles using quadratic interception mathematics, and the behaviour of each shot varies depending on the active weapon's speed, accuracy, and fire rate. I used the existing steering codebase from previous tasks (seek, physics integration, and Vector2D math) as the foundation and built the targeting system on top of it.

Simulation Setup

I created two distinct agents: a static **AttackerAgent** positioned on the left side of the screen, and a **TargetAgent** that patrols between two waypoints on the right side. The target's movement uses the same seek steering behaviour from Task 13 — each frame it calculates a desired velocity toward the current waypoint, applies the steering force through the standard $F=ma$ physics loop, and switches to the next waypoint when it arrives within 20px.

Each agent tracks:

- **pos** — current world position
- **vel** — current velocity
- **heading** — normalised direction vector derived from velocity

Projectile System

I created a `Projectile` class that spawns at the attacker's position with a velocity vector pointing toward the predicted interception point. Each frame, the projectile moves forward using Euler integration ($pos += vel * dt$) and checks for a hit by comparing its distance to the target against a fixed `HIT_RADIUS` of 15px. On a hit, the projectile deactivates and disappears, and the target flashes red for 0.3 seconds as visual confirmation. Projectiles that travel off screen are also cleaned up each frame to avoid memory buildup. The attacker fires on a cooldown timer driven by the active weapon's fire rate. At cooldown zero, a new projectile is spawned and the cooldown resets to $1.0 / fire_rate$ seconds.

Weapon Profiles

I defined four weapon profiles, each as a `WeaponProfile` object with three properties: projectile speed (pixels per second), accuracy (0.0 to 1.0), and fire rate (shots per second).

Weapon	Speed	Accuracy	Fire Rate
Rifle	600 px/s	98%	2/s
Rocket	150 px/s	97%	0.5/s
Handgun	500 px/s	75%	3/s
Hand Grenade	120 px/s	70%	0.3/s

The player can switch between weapons using keys 1–4 at runtime. The active weapon's name, speed, accuracy, and fire rate are displayed in the top-left corner of the window.

Target Prediction

The core of this spike is the quadratic interception calculation inside `calculate_interception()`. Rather than aiming at the target's current position, the attacker solves for the future time t at which a projectile travelling at a fixed speed will meet the target's projected position.

The problem reduces to a quadratic equation $at^2 + bt + c = 0$, where:

```

a = tv.dot(tv) - projectile_speed2
b = 2 * rel_pos.dot(tv)
c = rel_pos.rel_pos

```

Here `tv` is the target's velocity and `rel_pos` is the vector from the attacker to the target. The discriminant $b^2 - 4ac$ determines whether an interception is possible, a negative discriminant means the target is moving faster than the projectile and can never be caught. If a solution exists, I take the smallest positive root as the interception time t , then compute the predicted point as `target.pos + target.vel * t`.

If no valid t is found, the attacker falls back to aiming at the target's current position. This prediction is most visually obvious with the Rocket weapon. Because the projectile is slow, the interception point sits well ahead of the target's current position, the attacker is genuinely thinking ahead rather than reacting.

Weapon Inaccuracy

To differentiate weapons beyond speed and fire rate, I applied a random angular deviation to each shot based on the weapon's accuracy. The maximum spread angle scales as $(1.0 - \text{accuracy}) \times 30^\circ$, so a Rifle at 98% accuracy deviates by at most 0.6° , while a Hand Grenade at 70% accuracy can deviate by up to 9° .

The deviation is applied by rotating the normalised aim direction vector by a random angle drawn from the range `[-spread, +spread]` using the standard 2D rotation formula:

```

new_x = dir.x * cos(angle) - dir.y * sin(angle)
new_y = dir.x * sin(angle) + dir.y * cos(angle)

```

Observed Weapon Behaviours

Switching between weapons makes the differences immediately visible:

- **Rifle** — fast, tight shots that track close to the predicted interception point. The target is hit frequently, and the shot spread is barely noticeable.
- **Rocket** — rare, slow projectiles with a long travel time. The predicted interception crosshair sits far ahead of the target, demonstrating the prediction system working at its most extreme. A missed Rocket shot is obvious because the projectile visibly travels across the screen.
- **Handgun** — rapid shots with visible spread. A cone of projectiles fans out from the attacker, some hitting and some missing. The high fire rate means the target is still hit regularly despite the inaccuracy.
- **Hand Grenade** — very rare, slow, and wildly scattered shots. Most miss entirely. This weapon makes the contrast between accurate prediction and random spread most apparent.
- **Pressing [I]** at any time reveals the debug overlay: a red line from the attacker to the predicted interception point, and a green crosshair marking that point. This directly visualises that the attacker is leading the target, not following it.

Extension 1: Dynamic Range Finding and Close Quarters Combat

I added a fifth weapon, the Shotgun, and gave the attacking agent the ability to move. The Shotgun has a very short effective range of 150px, fires 8 pellets per shot in a 20° spread cone, and has a low fire rate of 0.7 shots per second. At long range it is useless, so the attacker must close the distance before engaging.

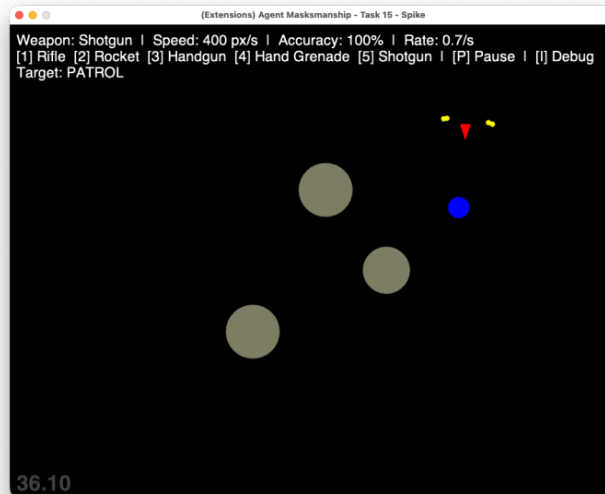
To support this, I added movement physics to **AttackerAgent** — velocity, heading, mass, and a max speed of 180 px/s. The `update()` method now branches on the weapon's `effective_range` property. When the Shotgun is selected and the distance to the target exceeds 150px, the attacker applies a seek steering force toward the target and moves. Once within range, it brakes to a stop and begins firing. The `_fire_shotgun()` method fires all 8 pellets in a single burst. Each pellet uses the same quadratic interception calculation as the baseline weapons to find the predicted aim point, then applies an independent random angular rotation within the spread cone:

```

angle = random.uniform(-spread_angle_maximum, +spread_angle_maximum)

```

The outcome is clearly distinct from the baseline weapons — the attacker visibly closes the distance before engaging, then unleashes a burst that can register multiple hits on the target in a single shot.



Extension 2: Area of Effect Detonation and Proximity Targeting

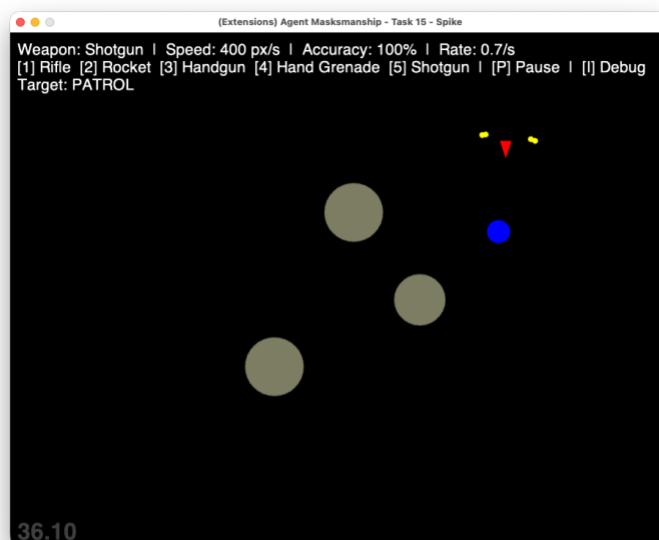
I modified the **Hand Grenade** to fire an `ExplosiveProjectile` rather than a standard direct hit projectile and updated the attacker's targeting logic to aim at the target's proximity rather than its body.

`ExplosiveProjectile` inherits from `Projectile` and adds a `blast_radius` of 80px and a detonation trigger. Rather than registering a hit on contact, it travels to a predetermined `aim_point` and detonates when it arrives within 25px of that point. On detonation, it checks if the target is within the 80px blast radius and registers a hit if so. A brief orange ring is drawn at the detonation point for 0.5 seconds as visual confirmation.

The proximity targeting logic in `_fire_explosive()` shifts the aim point 30px below the predicted body intercept, directing the projectile toward the target's feet rather than its centre:

```
aim_point = predicted_body_pos + Vector2D(0, -PROXIMITY_OFFSET)
```

This means the grenade does not need a direct body hit to deal damage — the AoE radius compensates for the offset. The green debug crosshair sits slightly below the target's centre, visually demonstrating that the attacker is deliberately aiming at the proximity point rather than the body.



Extension 3: Evasive Steering Behaviours and Cover Seeking Logic

I added three static circular obstacles to the world and gave the **TargetAgent** a state machine with three states — **PATROL**, **EVADE**, and **SEEK_COVER** — that activate based on incoming projectile threats.

Each frame, `_detect_threat()` scans active projectiles and flags any that are slow (below 200 px/s) and within a 200px detection radius. Fast projectiles such as the Rifle and Handgun are deliberately ignored — they are too fast to react to, so attempting to evade them is not meaningful. `ExplosiveProjectile` instances are also excluded since a detonating blast cannot be dodged.

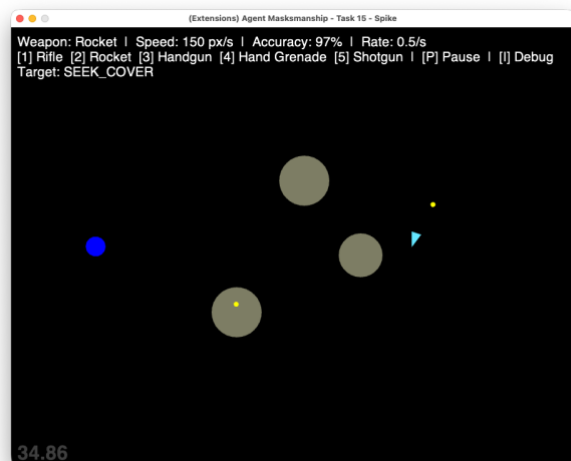
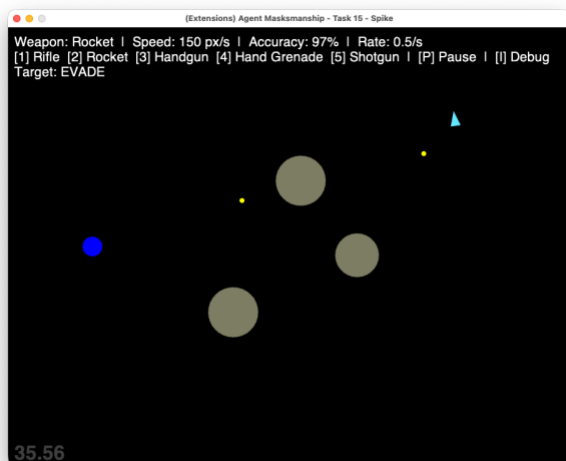
When a threat is detected, the target checks whether the nearest obstacle is within a 220px threshold:

- **Within 220px** → **SEEK_COVER**: the target uses `_calculate_cover_pos()` to find the point directly behind the obstacle relative to the attacker's position, then uses an arrive steering behaviour to move smoothly to that point and stop.
- **Beyond 220px** → **EVADE**: the target applies a flee force directly away from the incoming projectile's current position.
- **No threat** → **PATROL**: normal waypoint seeking resumes.

The cover position is calculated by projecting the attacker-to-obstacle direction through the obstacle and placing the hide point behind it:

```
direction = normalise(obstacle.pos - attacker.pos)
safe_point = obstacle.pos + direction * (obstacle.radius + padding)
```

Obstacle avoidance is applied as an additional push force in all three states to prevent the target from clipping into cover geometry while fleeing. The target's current state is shown live in the UI, making the transitions between states immediately visible when switching to the Rocket or Hand Grenade weapon.



Conclusion

I built a complete agent marksmanship simulation covering intelligent targeting, weapon diversity, and active target response. The baseline demonstrated that a single quadratic formula is sufficient to make an attacker feel genuinely intelligent. It leads the target rather than reacting to it, and the difference is most visible with slow weapons like the Rocket.

The extensions progressively increased complexity on both sides of the encounter. The Shotgun extension introduced movement-based decision making where the attacker dynamically closes the distance before engaging, turning effective range into an active AI variable rather than just a stat. The AoE extension showed that accurate targeting does not always mean hitting the body, aiming at proximity and letting blast radius do the work is a more effective strategy for slow explosive weapons. The evasion extension shifted the challenge to the target, demonstrating that even a simple three-state machine produces believable reactive behaviour when combined with well-tuned detection thresholds and spatial cover calculations.

The key insight from this spike is that marksmanship AI is fundamentally a prediction problem — every decision, from leading a moving target to aiming at the ground near its feet to detecting a slow incoming threat, comes down to reasoning about where things will be in the near future rather than where they are right now.